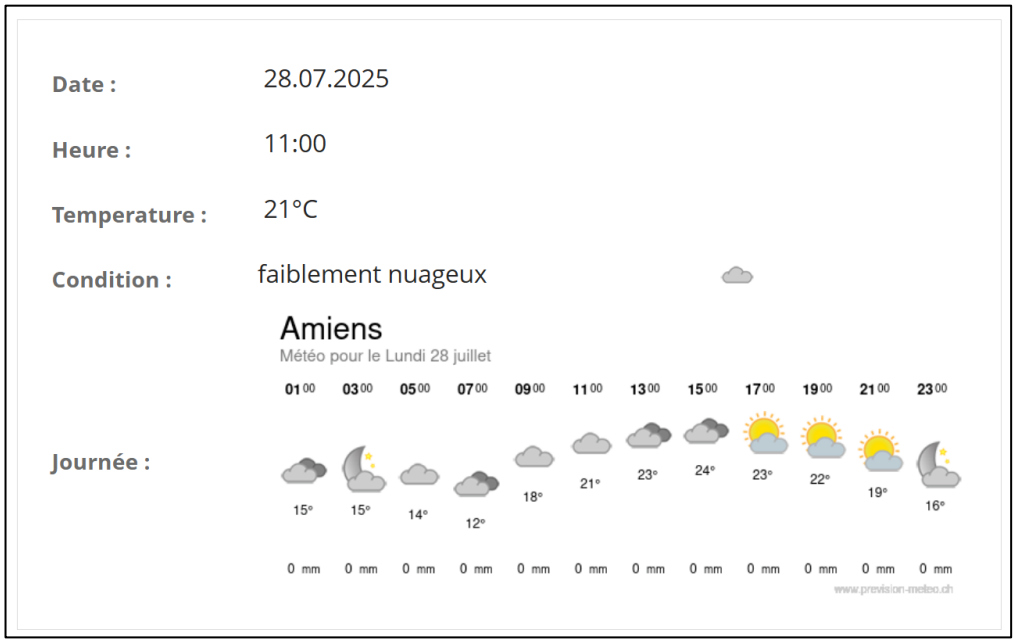
Objectif de l'activité professionnelle

A la fin de cette activité, vous serez capable :

D'utiliser des API externes publiques De coupler plusieurs API pour répondre à un besoin.

1. Version 1 : répertoire meteo/amiens

La première version doit simplement afficher la météo du jour sur la ville d'Amiens.



Pour cela il faut utiliser l'API le service Web offert par le site *https://www.prevision-meteo.ch/*
La documentation du web service est consultable à l'adresse : *https://prevision-meteo.ch/services*
Un PDF est fourni : https://www.prevision-meteo.ch/uploads/pdf/recuperation-donnees-meteo.pdf

Dans cette première version, il faut simplement interroger l'url :
https://www.prevision-meteo.ch/services/json/amiens


```
{
  - city_info: {
        name: "Amiens",
        country: "France",
        latitude: "49.8944435",
        longitude: "2.2933331",
        elevation: "35",
        sunrise: "08:00",
        sunset: "18:09"
    },
  - forecast_info: {
        latitude: null,
        longitude: null,
        elevation: "67.4"
    },
  - current_condition: {
        date: "14.02.2021",
        hour: "14:00",
        tmp: 0,
        wnd_spd: 27,
        wnd_gust: 0,
        wnd_dir: "SE",
        pressure: 1028.1,
        humidity: 51,
        condition: "Ensoleillé",
        condition_key: "ensoleille",
        icon: "https://prevision-meteo.ch/style/images/icon/ensoleille.png",
        icon_big: "https://prevision-meteo.ch/style/images/icon/ensoleille-big.png"
    },
  - fcst_day_0: {
        date: "14.02.2021",
        day_short: "Dim.",
        day_long: "Dimanche"
```


L'image associée sur la météo de la journée s'obtient par l'url suivante https://www.prevision-meteo.ch/uploads/widget/amiens_0.png

L'image étant connue, elle est définie directement dans le script index.html

```
<td>
   <img id="journee"
src="https://www.prevision-meteo.ch/uploads/widget/amiens_0.png"
            width="650" height="250"/>
</td>
```

Dans le fichier index.js, il faut réaliser l'appel de l'API en utilisant la version simplifiée d'un appel Ajax appelAjaxSimple() car il ne faut pas ajouter dans la requête le moindre entête comme le fait la fonction appelAjax avec l'entête 'X-Requested-With'.

La raison : Lorsqu'une requête fecth comporte un header, le navigateur déclenche automatiquement une requête OPTIONS préalable (preflight) afin de savoir si le serveur accepte la requête avec ses entêtes or le serveur www.prevision-meteo.ch n'accepte visiblement pas ce genre de requête pré-volée, ce qui provoque une erreur CORS silencieuse, la requête est alors bloquée côté navigateur, avant même d'atteindre le serveur.

En envoyant une requête simple sans entête, le navigateur n'interroge pas le serveur à l'aide d'une requête option et laisse passer directement la requête.

```
appelAjaxSimple({
    url: 'https://www.prevision-meteo.ch/services/json/amiens',
    success: afficherMeteo
})
```

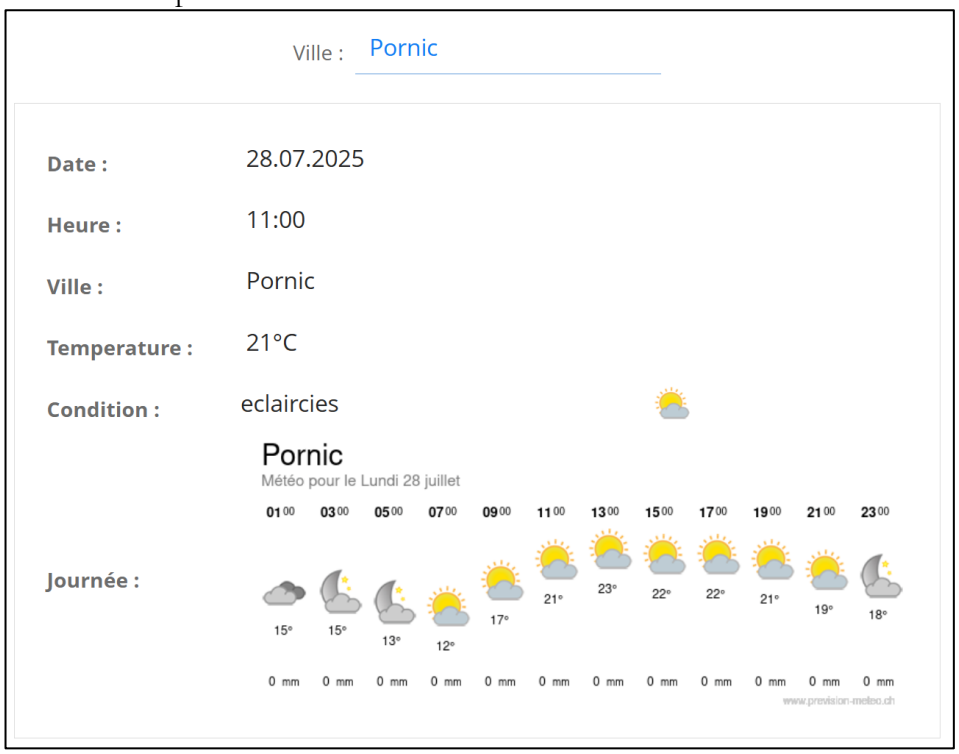
La fonction afficherMeteo() doit alimenter chaque zone nommée de l'interface en cherchant leur correspondance dans la réponse obtenue au format json
Par exemple la date se trouve dans la propriété data.current_condition.date

```
function afficherMeteo(data) {
    date.innerText = data.current_condition.date
    heure.innerText = data.current_condition.hour
    temperature.innerText = data.current_condition.tmp + "°C";
    condition.innerText = data.current_condition.condition.toLowerCase()
    icone.src = data.current_condition.icon;
}
```

1. Compléter le script meteo/amiens/index.js pour obtenir l'affichage demandé.

2. Version 2 : répertoire meteo/ville

Cette version permet de sélectionner la ville souhaitée :



L'appel Ajax vers l'API doit s'effectuer sur le changement de valeur de la balise 'search'.
Il faut préalablement retirer les accents éventuellement saisis en utilisant la fonction enleverAccent du module format.js et mettre la valeur en minuscule.
La réponse sera traitée par la fonction afficherMeteo

Il est possible que la ville sélectionnée ne soit pas référencée dans le service web météo. Dans ce cas le service retourne toujours une réponse JSON mais de la forme suivante :

```
{
  - errors: [
      - {
            code: "11",
            text: "City or coordinate not found",
            description: "Please specify a valid city or a coordinate. Available cities are listed here : http://www.prevision-meteo.ch/services/json/list-cities"
        }
    ]
}
```

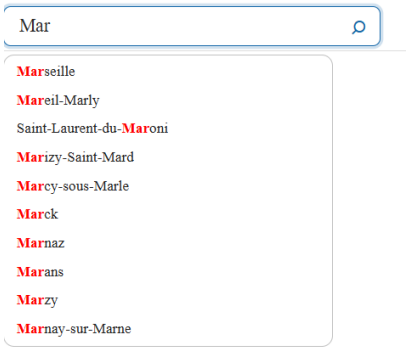
La fonction afficherMeteo(data) doit donc tester si le propriété errors afin d'afficher l'erreur(le texte et la description) à l'aide de la méthode messageBox de la bibliothèque afficher.js

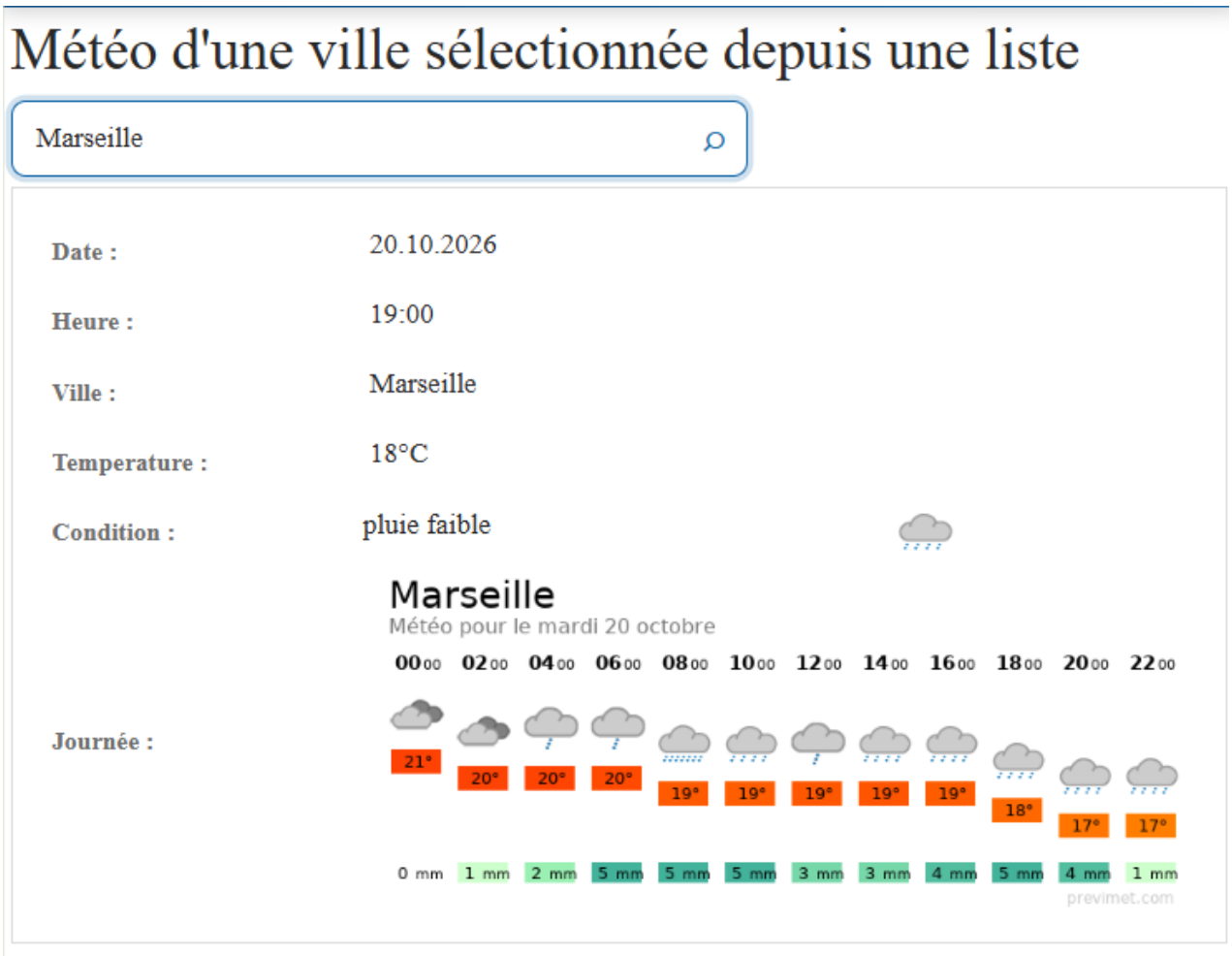
En cas de succès, la fonction afficherMeteo(data) est toujours chargée d'alimenter les zones de l'interface.
La source de l'image de la journée est maintenant dynamique (data.city_info.name) mais là encore il faut retirer les accents et mettre la valeur en minuscules

2. Compléter le script meteo/ville/index.js et réaliser les tests fonctionnels

3. Version 3 : répertoire meteo/autocompletion

La troisième version ajoute un système d'autocomplétion afin de pouvoir sélectionner une ville.





Le système d'autocomplétions est mis en place à l'aide du composant autocomplete

```
<link rel="stylesheet" href="/css/autocomplete.css">
<script src="https://cdn.jsdelivr.net/npm/@tarekraafat/autocomplete.js@10.2.10/dist/autoComplete.min.js"></script>
```

Pour alimenter la source de données de notre système d'autocomplétion nous utilisons une nouvelle API qui permet de récupérer les villes françaises :
https://geo.api.gouv.fr/decoupage-administratif/communes

Exemple d'appel :
https://geo.api.gouv.fr/communes?fields=nom&boost=population&nom=marseille

Nous récupérons dans le format json un tableau comprenant toutes les villes contenant la chaine 'marseille'.

Chaque objet 'ville' possède les propriétés suivantes : nom, code, codeDepartement, siren, codeEpci, codeRegion, codesPostaux (un tableau), population

Deux paramètres supplémentaires peuvent être ajoutés :

- Le paramètre **boost=population** permet de trier les résultats de l'API par ordre décroissant de population. Cela ajoute dans la réponse une propriété _score représentant le score de pertinence.
- Le paramètre fields permet de restreindre les champs renvoyés dans la réponse JSON, ce qui limite le volume des données transférées et pas conséquence réduit le temps de traitement. Le propriété code sera toujours présente dans la réponse car elle sert d'identifiant.

Nous souhaitons simplement récupérer le nom de la ville.

La fonction rechercherMeteo(ville) remplace l'événement onChange de la version précédente. Elle doit enlever les accents au paramètre 'ville' et appeler l'API ('https://www.prevision-meteo.ch/services/json/' + ville) à l'aide de la fonction appelAjaxSimple

La réponse sera traitée par la fonction afficherMeteo
La fonction afficherMeteo reste identique à la version précédente.

Pour paramétrer le composant autocomplete, consulter la base de connaissances Obsidian :
Bloc 2 > 02 Développement Web > 03 Éléments pratiques > Composant Client > autocomplete.js

3. Compléter le script meteo/autocompletion/index.js et réaliser les tests fonctionnels

4. Version 4 : répertoire meteo/geolocalisation

La dernière version doit afficher la météo en fonction de la position de l'utilisateur.

La méthode navigator.geolocation.getCurrentPosition est utilisée pour obtenir la position géographique actuelle de l'utilisateur. Cette méthode prend deux fonctions de rappel (callback) : une pour le succès et une pour l'erreur.

```
navigator.geolocation.getCurrentPosition(success, erreur);
```

Si à partir des coordonnées récupérées nous utilisons l'api :

https://www.prevision-meteo.ch/services/json/lat=49.8944435lng=2.2933331

Nous remarquons que l'api ne récupère pas le nom de la ville


```
{
 "city_info": {
  "name": "NA",
  "country": "--",
  "latitude": "49.8944435",
  "longitude": "2.2933331",
```


Pour récupérer le nom de la ville il faut passer par l'API de reverse geocoding

https://api-adresse.data.gouv.fr/reverse/?lon=2.2933331&lat=49.8944435

Nous obtenons alors toutes les adresses proches de cette position


```
{
 "type": "FeatureCollection",
 "features": [
  {
   "type": "Feature",
   "geometry": {
    "type": "Point",
    "coordinates": [2.293375, 49.894426]
   },
   "properties": {
    "type": "housenumber",
    "name": "4 Rue du Général Leclerc",
    "label": "4 Rue du Général Leclerc 80000 Amiens",
    "street": "Rue du Général Leclerc",
    "postcode": "80000",
    "citycode": "80021",
    "city": "Amiens",
    …
  }
```


Toutes ces adresses concernent forcément la même ville, nous pouvons donc récupérer la première valeur de l'attribut city : data.features[0].properties.city;

Nous avons donc besoin, une fois les coordonnées GPS récupérés d'appeler une fonction qui va rechercher la ville correspondante via un appel Ajax de l'api reverse.

Il est possible que les coordonnées GPS récupérées ne donne pas de résultat

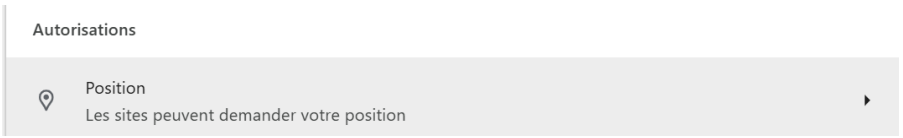
```
function rechercherVille(position) {
    const lat = position.coords.latitude;
    const lng = position.coords.longitude;
    appelAjaxSimple({
        url: "https://api-adresse.data.gouv.fr/reverse/?lat=" + lat + "&lon=" + lng,
        success: (data) => {
            if (data && data.features && data.features.length > 0) {
                rechercherMeteo(data.features[0].properties.city);
            } else {
                messageBox("Aucune ville trouvée pour cette position.", 'error');
            }
        }
    });
}
```

La fonction rechercherMeteo reste identique à la version précédente.
La fonction afficherMeteo reste identique à la version précédente.

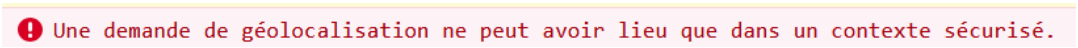
Si l'on teste notre interface nous obtenons dans la console le message : L'utilisateur a refusé la demande

En effet pour pouvoir récupérer les coordonnées GPS du visiteur, plusieurs conditions doivent être réunies :

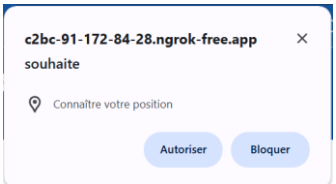
La configuration du navigateur doit autoriser la détection de la position : c'est le cas normalement
Pour le vérifier depuis le navigateur : Paramètres – Confidentialité et sécurité – Paramètres des sites



La demande doit être réalisée en utilisant le protocole HTTPS



L'utilisateur doit accepter la demande



Nous devons donc réussir à lancer notre module en Https, or WampServer n'installe pas ce protocole par défaut.
Il est normalement possible de mettre en place le protocole HTTPS en utilisant un certificat SSL auto-signé pour chaque Virtual host. La procédure est longue et ne fonctionne pas toujours.

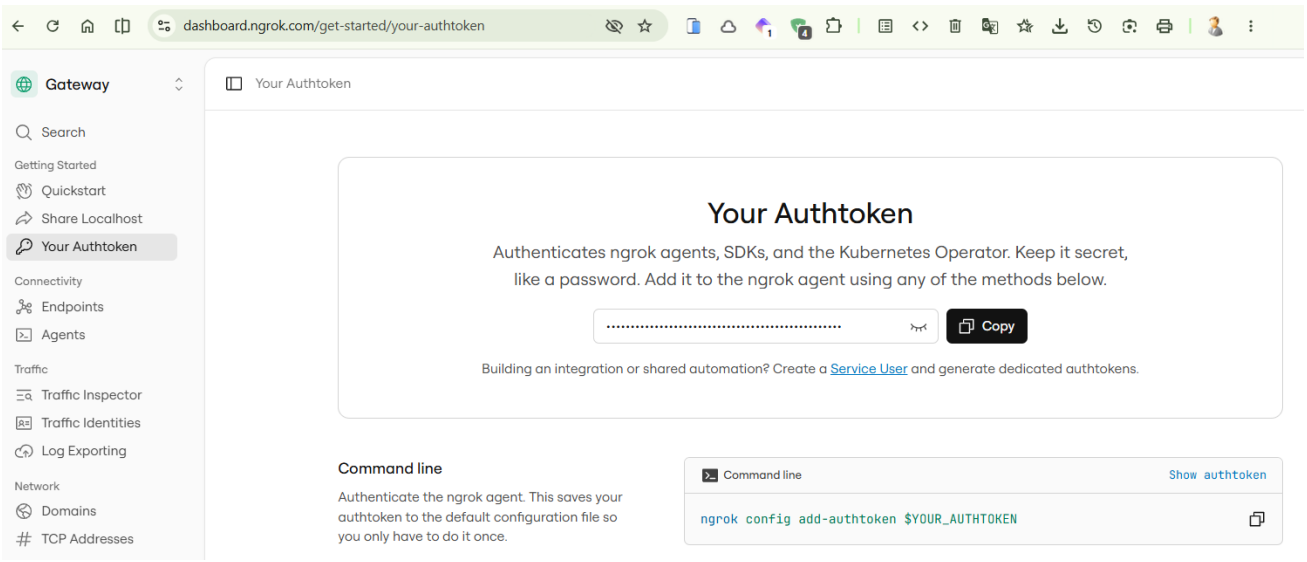
Une autre solution plus simple à mettre en place consiste à utiliser l'outil **ngrok**.
ngrok est un outil qui permet de créer des tunnels sécurisés vers des serveurs locaux, facilitant ainsi l'accès à des applications locales depuis l'extérieur. Il est particulièrement utile pour les développeurs qui souhaitent partager des applications en cours de développement avec des collègues sans avoir à configurer des serveurs publics.

ngrok crée un tunnel sécurisé entre votre machine locale et un serveur public.
Le service fournit alors une URL publique utilisant le protocole HTTPS.
Par exemple : https://c2bc-91-172-84-28.ngrok-free.app/meteo/geolocalisation/index.php

Vous devez commencer par créer votre compte sur la plateforme ngrok : https://ngrok.com/.
Vous pouvez utiliser votre compte GitHub pour cela.

Se rendre ensuite dans la section "Your Authtoken" de votre tableau de bord ngrok et copier le token d'authentification (ajouter le dans votre coffre-fort).

https://dashboard.ngrok.com/get-started/your-authtoken



Il faut maintenant installer l'utilitaire client. Plusieurs solutions sont proposées depuis la page de téléchargement : https://ngrok.com/download/windows?tab=download

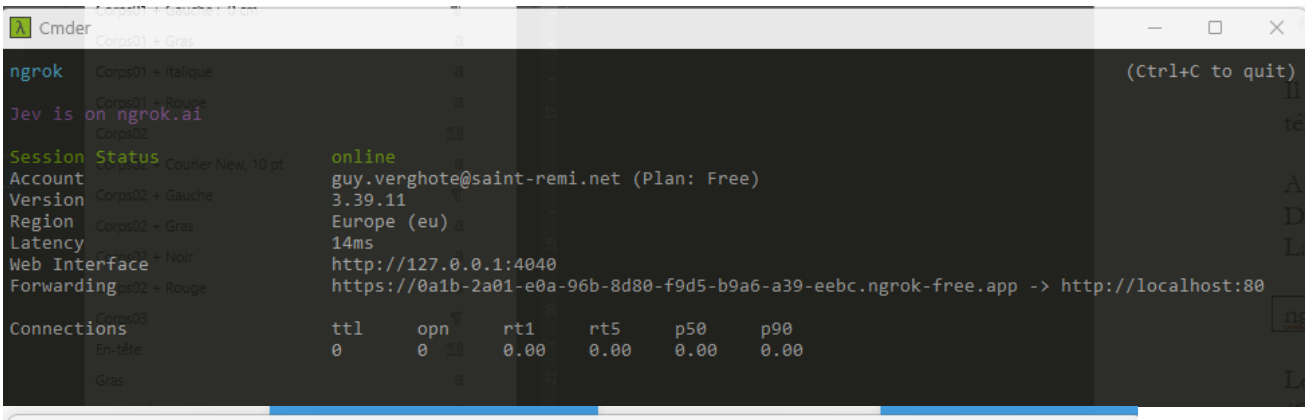
Afin de gagner du temps l'utilitaire ngrok.exe est disponible sur J:\Utilitaire
Démarrer une console Cmder dans le dossier de ce fichier
Lancer ensuite la commande suivante pour vous authentifier :

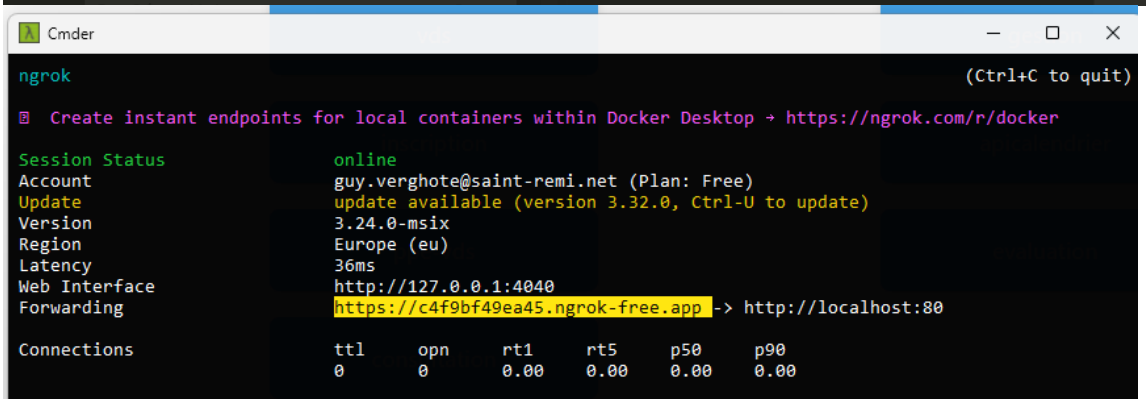
```
ngrok authtoken YOUR_AUTH_TOKEN
```

Le token est alors stocké dans votre profil (C:\Users\nomUser\AppData\Local/ngrok/ngrok.yml), il n'est donc plus nécessaire de relancer cette commande par la suite.

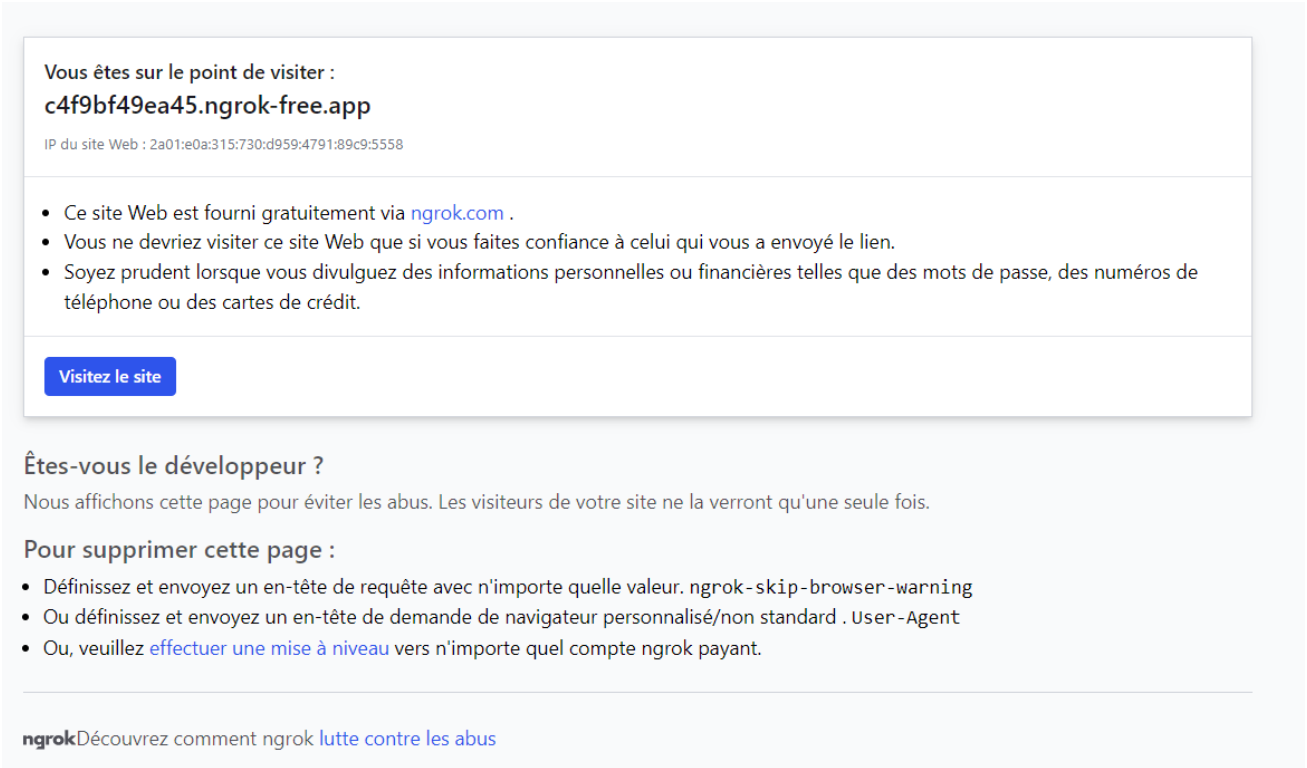
Lancer la commande suivante pour créer le tunnel et obtenir une url publique utilisant HTTPS

```
ngrok http --host-header=api 80
```





Ouvrir l'URL dans votre navigateur (dans l'exemple : https://8ce2a473faa8.ngrok-free.app/)



Lancer le module meteo/geolocalisation

A l'ouverture de la page, le navigateur vous demande une autorisation



Date : 21.10.2026

Heure : 07:00

Ville : Amiens

Temperature : 9°C

Condition : fortement nuageux

Journée :

